1. Introducción

En esta actividad continuamos con el desarrollo de un prototipo de una ambulancia, el cual posteriormente tendrá diferentes funciones para poder controlar su movimiento y brindar seguridad. La idea principal del proyecto es que el carrito pueda ser controlado de manera independiente en cada uno de sus lados para lograr giros, mientras monitorea su entorno.

En la primera etapa nos enfocamos en el sistema motriz, utilizando un Arduino UNO, un controlador de motores L293D, botones y motores. Los botones funcionan como la forma en la que el usuario le indica al carrito qué movimiento realizar, y logramos que las llantas pudieran moverse hacia adelante, hacia atrás y realizar giros. En este documento corregimos un error de la entrega anterior, donde se mencionó erróneamente que solo se controlaba el lado izquierdo; en realidad, el sistema de tracción para ambos lados ya estaba funcional.

En esta segunda etapa, el enfoque principal fue agregar un sensor de distancia junto con un buzzer para crear una alerta de proximidad. La intención es que el carrito pueda detectar si tiene un objeto cerca y avisar mediante sonidos, en vez de depender solamente de que la persona esté calculando la distancia a simple vista. Mientras más cerca se encuentre el objeto, más seguido sonará el buzzer, para que la alerta sea más clara. De esta manera, el prototipo incluye una función de apoyo para evitar choques, la cual fue optimizada en el código para funcionar simultáneamente sin interrumpir el movimiento del carrito.

2. Objetivo

El objetivo de esta actividad es desarrollar la segunda parte del prototipo de ambulancia, enfocándonos en incorporar un sistema de aviso de colisiones de manera fluida, usando un sensor de distancia y un buzzer, y sumándolo al sistema de movimiento que ya teníamos.

El sensor permitirá reconocer qué tan cerca está un objeto y el buzzer dará una señal sonora cuya frecuencia aumentará conforme disminuya la distancia. Con esto se busca que el carrito tenga una función de asistencia y que el usuario pueda reaccionar a tiempo si se acerca demasiado a un obstáculo. Un objetivo clave en esta etapa es utilizar técnicas de programación (como el uso de temporizadores internos) que permitan que el buzzer suene intermitentemente sin pausar ni afectar el funcionamiento de los motores controlados por el L293D.

3. Materiales

Para realizar esta parte del proyecto se utilizaron los siguientes materiales:

- 1 Arduino UNO
- 1 controlador de motores L293D
- 4 motores de corriente directa
- 4 botones pulsadores
- 4 resistencias
- 1 protoboard
- Cables jumper
- Fuente de alimentación
- Computadora para programar el Arduino
- Cable USB para conectar el Arduino a la computadora

- 1 sensor de distancia ultrasónico (HC-SR04) para detectar objetos
- 1 buzzer (piezoeléctrico) para generar las alertas sonoras

4. Metodología

Al iniciar el sistema, el Arduino está programado para revisar de manera constante el estado de los cuatro botones mediante la lectura digital. El usuario interactúa presionando los botones para indicarle al prototipo la acción que debe realizar. Al presionar el botón para avanzar o retroceder, el Arduino envía la señal correspondiente al controlador L293D, el cual establece la dirección y acciona los motores. Si no se detecta ningún botón presionado, se ejecuta la instrucción para apagar los motores. Simultáneamente, si el sensor detecta un objeto cerca, el buzzer se activa.

4.1 Conexión de los botones

Se utilizaron cuatro botones pulsadores y cuatro resistencias, los cuales fueron colocados en la protoboard. Las conexiones hacia el Arduino se distribuyeron en los pines digitales para poder recibir la señal correspondiente a cada movimiento:

- Botón Izquierda: Conectado como entrada al pin 2.
- Botón Atrás: Conectado como entrada al pin 3.
- Botón Adelante: Conectado como entrada al pin 4.
- Botón Derecha: Conectado como entrada al pin 5.

4.2 Funcionamiento y conexión de los motores y el controlador L293D

Los motores de corriente directa (DC) funcionan girando en un sentido u otro dependiendo de la polaridad de la energía que reciben. Para poder controlar este giro desde el Arduino sin tener que cambiar los cables físicamente, se implementó un controlador de motores L293D. Este circuito integrado funciona internamente como un puente H, lo que permite cambiar la dirección en la que fluye la corriente y, por lo tanto, decidir si el motor gira hacia adelante o hacia atrás. Además, el L293D es necesario porque aísla la etapa de control (el Arduino) de la etapa de potencia; los motores requieren más corriente de la que el Arduino puede entregar, por lo que el controlador toma la energía de una fuente de alimentación externa y la dirige a los motores de forma segura.

Para lograr controlar las llantas izquierdas, los pines del controlador L293D se conectaron de la siguiente manera:

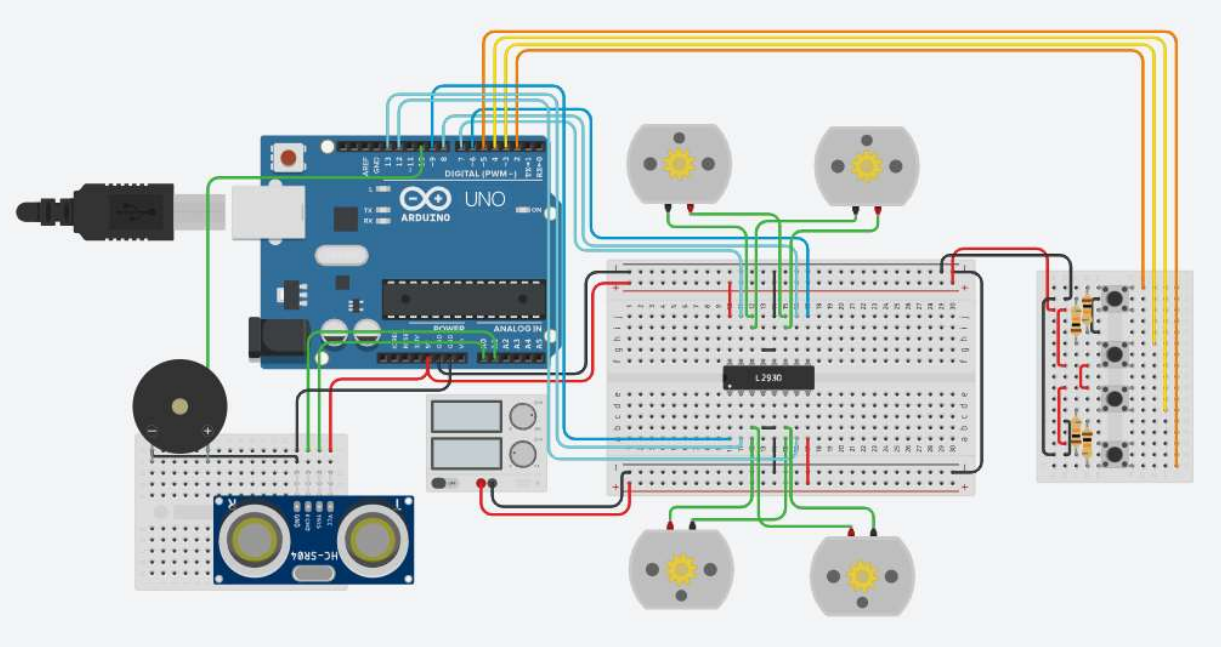
- **Pin Enable (Habilitador 1,2):** Se conectó al puerto digital 6 del Arduino (definido en el código como motorEna). Este pin sirve para encender o apagar el canal del motor y nos permite controlar la velocidad utilizando modulación por ancho de pulso (PWM).
- **Pines Input 1 e Input 2 (Entradas de control):** Se conectaron a los puertos 7 (motorIn1) y 8 (motorIn2) del Arduino, respectivamente. Estos pines reciben las instrucciones lógicas. Como se observa más adelante en el código, si motorIn1 está en estado HIGH y motorIn2 en LOW, el motor gira hacia adelante. Si se invierte y motorIn1 es LOW mientras motorIn2 es HIGH, el giro es hacia atrás (dirección opuesta).
- **Pines Output 1 y Output 2 (Salidas al motor):** Estos pines se conectaron directamente a las terminales de los motores de corriente directa del lado izquierdo. Son los encargados de transmitir el voltaje con la polaridad correcta dictada por las entradas.
- **Pines GND (Tierra):** Se conectaron a la línea negativa de la protoboard para unificar las tierras del Arduino, la fuente de alimentación externa y el controlador L293D, cerrando así el circuito.
- **Pines de voltaje (Vcc1 y Vcc2):** Uno de estos pines se conecta a los 5V del Arduino para la alimentación lógica del chip, y el otro se conecta a la línea positiva de la fuente de alimentación externa, la cual es la encargada de suministrar la energía con mayor potencia directamente a los motores.

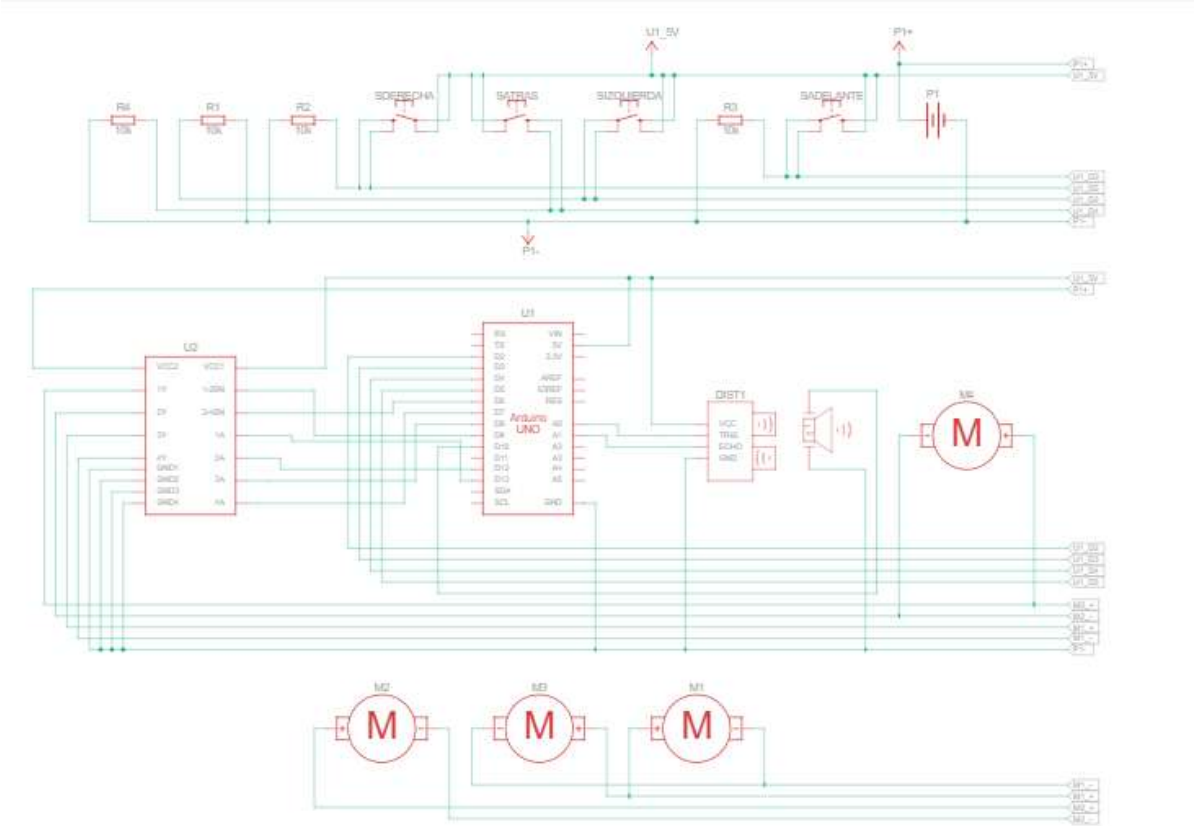
4.3 Conexión del sensor y el buzzer

Con el fin de optimizar las conexiones dejando libres los pines que requieren PWM para otros componentes, se realizaron los siguientes ajustes:

- **Sensor (TRIG y ECHO):** Se conectaron a los puertos analógicos A0 y A1 del Arduino respectivamente. Estos puertos pueden funcionar perfectamente como pines digitales para enviar y recibir los pulsos del sensor ultrasónico.
- **Buzzer:** Se conectó al puerto digital 10. Es fundamental usar un puerto con soporte PWM (como el 10) para emitir la señal eléctrica adecuada que el piezoeléctrico convierte en sonido.
-

4.3 Diagrama de conexión





5. Código utilizado

```ino
// --- PINES DE LOS BOTONES ---
int botonIzquierda = 2;
int botonAtras = 3;
int botonAdelante = 4;
int botonDerecha = 5;

// --- MOTORES LADO IZQUIERDO (Canal 1) ---
int motorEna = 6; // Control de velocidad Izquierda (PWM)
int motorIn1 = 7; // Control de dirección Izquierda 1
int motorIn2 = 8; // Control de dirección Izquierda 2

// --- MOTORES LADO DERECHO (Canal 2) ---
int motorEnb = 9;  // Control de velocidad Derecha (PWM)
int motorIn3 = 12; // Control de dirección Derecha 1
int motorIn4 = 13; // Control de dirección Derecha 2

// --- PINES SENSOR ULTRASÓNICO Y BUZZER ---
const int buzzerPin = 10; // BUZZER
const int trigPin = A0;   // TRIG
const int echoPin = A1;   // ECHO

// Velocidad base de los motores (0 a 255)
int velocidad = 255;

// Variables para el control del buzzer
unsigned long tiempoAnteriorBuzzer = 0;
bool estadoBuzzer = false;
```

```
void setup() {
  pinMode(botonIzquierda, INPUT);
  pinMode(botonAtras, INPUT);
  pinMode(botonAdelante, INPUT);
  pinMode(botonDerecha, INPUT);

  pinMode(motorEna, OUTPUT);
  pinMode(motorIn1, OUTPUT);
  pinMode(motorIn2, OUTPUT);

  pinMode(motorEnb, OUTPUT);
  pinMode(motorIn3, OUTPUT);
  pinMode(motorIn4, OUTPUT);

  // Configurar pines del Sensor y Buzzer
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(buzzerPin, OUTPUT);

  // Iniciar con todos los motores y el buzzer apagados
  digitalWrite(motorIn1, LOW);
  digitalWrite(motorIn2, LOW);
  analogWrite(motorEna, 0);

  digitalWrite(motorIn3, LOW);
  digitalWrite(motorIn4, LOW);
  analogWrite(motorEnb, 0);
```

```

  noTone(buzzerPin);
}

void loop() {

  long duracion;
  int distancia;

  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duracion = pulseIn(echoPin, HIGH, 30000);

  if (duracion == 0) {
    distancia = 100;
  } else {
    distancia = duracion * 0.034 / 2;
  }

//-----------------------------------------------------------------
  unsigned long tiempoActual = millis();

```

```
81
82    if (distancia > 0 && distancia <= 10) {
83      tone(buzzerPin, 1000);
84    }
85    else if (distancia > 10 && distancia <= 50) {
86      int intervaloBuzzer = map(distancia, 10, 50, 50, 500);
87
88      if (tiempoActual - tiempoAnteriorBuzzer >= intervaloBuzzer) {
89        tiempoAnteriorBuzzer = tiempoActual;
90        estadoBuzzer = !estadoBuzzer;
91
92        if (estadoBuzzer) {
93          tone(buzzerPin, 1000);
94        } else {
95          noTone(buzzerPin);
96        }
97      }
98    }
99    else {
100     noTone(buzzerPin);
101     estadoBuzzer = false;
102   }
103
```

```
104 // ---------------------------------------------------------------
105   bool estadoIzquierda = digitalRead(botonIzquierda);
106   bool estadoAtras     = digitalRead(botonAtras);
107   bool estadoAdelante  = digitalRead(botonAdelante);
108   bool estadoDerecha   = digitalRead(botonDerecha);
109
110   if (estadoAdelante == HIGH) {
111     digitalWrite(motorIn1, HIGH);
112     digitalWrite(motorIn2, LOW);
113     analogWrite(motorEna, velocidad);
114
115     digitalWrite(motorIn3, HIGH);
116     digitalWrite(motorIn4, LOW);
117     analogWrite(motorEnb, velocidad);
118   }
119   else if (estadoAtras == HIGH) {
120     digitalWrite(motorIn1, LOW);
121     digitalWrite(motorIn2, HIGH);
122     analogWrite(motorEna, velocidad);
123
124     digitalWrite(motorIn3, LOW);
125     digitalWrite(motorIn4, HIGH);
126     analogWrite(motorEnb, velocidad);
127   }
```

```
  else if (estadoIzquierda == HIGH) {
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, HIGH);
    analogWrite(motorEna, velocidad);

    digitalWrite(motorIn3, HIGH);
    digitalWrite(motorIn4, LOW);
    analogWrite(motorEnb, velocidad);
  }
  else if (estadoDerecha == HIGH) {
    digitalWrite(motorIn1, HIGH);
    digitalWrite(motorIn2, LOW);
    analogWrite(motorEna, velocidad);

    digitalWrite(motorIn3, LOW);
    digitalWrite(motorIn4, HIGH);
    analogWrite(motorEnb, velocidad);
  }
  else {
    digitalWrite(motorIn1, LOW);
    digitalWrite(motorIn2, LOW);
    analogWrite(motorEna, 0);

    digitalWrite(motorIn3, LOW);
    digitalWrite(motorIn4, LOW);
    analogWrite(motorEnb, 0);
  }

  delay(20);
}
```

6. Explicación del código

El código en Arduino comúnmente se divide en dos secciones, "setup" y "loop". En la sección setup, se ponen las instrucciones que se desean ejecutar únicamente una vez cuando el sistema inicie, y en loop, el que se desea repetir.

Lo primero que se hizo fue crear las variables de los pines. Se crearon cuatro para los botones (2,3,4 y 5) y seis para controlar el controlador L293D ("motorEna", "motorEnb", "motorIn1", etc.), las cuales reciben las instrucciones de hacia dónde girar. Después, se creó la variable "velocidad" con valor 255. Posteriormente se crearon las nuevas variables relacionadas al sensor de distancia y al buzzer ("trigPin" en A0, "echoPin" en A1 y "buzzerPin" en 10).

Dentro del setup, utilizamos la función pinMode para establecer los botones y el echoPin como entradas (INPUT), mientras que los pines del controlador L293D, el trigPin y el buzzerPin se configuraron como salidas (OUTPUT). También nos aseguramos de que el programa inicie con los motores en LOW (apagados) y el buzzer silenciado con noTone().

Para la sección loop, el código se dividió en tres tareas que ocurren casi simultáneamente.

Primeramente, se activa el sensor para emitir la onda ultrasónica. La señal de rebote se lee con duracion = pulseIn(echoPin, HIGH, 30000);. A esta función se le agregó el límite de 30000 microsegundos con el fin de que cuando el sensor no reciba respuesta, el programa no se quede congelado esperando, devolviendo un valor que convertimos matemáticamente en la variable "distancia".

Teniendo la distancia, pasamos a la lógica del buzzer. Se utilizó la variable tiempoActual = millis(); que guarda el tiempo que el Arduino lleva encendido. A diferencia de entregas anteriores donde se pensaba usar "delay", aquí implementamos millis() para que el tiempo entre cada sonido del buzzer pueda controlarse sin detener el programa principal. Con la función map(), el programa ajusta el intervalo de los pitidos: si la distancia se reduce de 50 cm a 11 cm, el tiempo entre pitidos baja de 500 ms a 50 ms. Si la distancia es 10 cm o menos, suena de forma continua indicando alerta máxima.

Finalmente, en la lógica de los motores, el programa lee las variables "estadoAdelante", "estadoAtras", etc. Dependiendo del botón que tenga estado HIGH, se configuran los pines motorIn para lograr que el carrito avance, retroceda, o haga giros invirtiendo el sentido de un lado con respecto al otro. Al no haber demoras en esta sección, las ruedas responden instantáneamente a las instrucciones del usuario mientras el buzzer hace su trabajo en segundo plano.

7. Resultados

Después de cargar el código y realizar conexiones, se realizaron distintas pruebas con el fin de comprobar si el prototipo funcionaba como se espera. Con ellas, primeramente pudo comprobarse que al presionar el botón de movimiento hacia adelante, efectivamente el motor giraba en un sentido, y que este cambia al presionar el botón de movimiento hacia atrás, y además, pudo comprobarse que si el prototipo va en movimiento hacia atrás, el buzzer no sonaba si un objeto estaba a más de 100 centímetros, que a menos de 100 centímetros iba sonando cada vez más rápido entre más disminuia su distancia con el objeto, y que si el movimiento no era hacia atrás este permanencia apagado, por lo que puede concluirse que el Arduino detecta los botones, el sensor y el buzzer correctamente y que se están enviando las señales necesarias.

Botón presionado	Resultado
Ninguno	El motor no se mueve y el buzzer se mantiene apagado
Adelante	El motor gira hacia un sentido y el buzzer permanece apagado
Atrás (sin objetos a 100 cm o menos)	El motor gira en un sentido contrario y el buzzer permanece apagado
Atras (objeto a 100cm)	El motor gira en un sentido contrario y el buzzer pita lentamente
Atras (objeto a 50cm)	El motor gira en un sentido contrario y el buzzer pita más rápido que anteriormente
Atras (objeto a 20cm)	El motor gira en un sentido contrario y el buzzer pita más rápido que anteriormente
Atras (objeto a 5cm)	El motor gira en un sentido contrario y el buzzer pita muy rápido
Izquierda	Aún no se implementa la función, no se mueve y el buzzer permanece apagado
Derecha	Aún no se implementa la función, no se mueve y el buzzer permanece apagado

Escenario de prueba	Resultado
Objeto a más de 50 cm	El buzzer permanece apagado
Objeto entre 11 y 50 cm	El buzzer pita de manera intermitente.
Objeto a 10 cm o menos	El buzzer emite un tono continuo de advertencia.

8. Conclusión

En este avance logramos consolidar el sistema del prototipo de ambulancia. Tras aclarar y confirmar el control total de ambos lados de las llantas que se tenía previamente utilizando el L293D, logramos integrar exitosamente la segunda capa de este proyecto: el sistema de seguridad y alertas.

Mejorar el código utilizando un temporizador asíncrono con millis() en lugar de demoras con delay() hizo que el proyecto tuviera una función de advertencia mucho más profesional.

Además, agregar el sensor de distancia y el buzzer hace que el proyecto tenga una función extra de seguridad al retroceder. La alerta sonora permite identificar cuándo hay un objeto cerca y, mientras menor sea la distancia, los pitidos se escuchan más seguido. Esto puede ayudar a evitar golpes durante las maniobras, aunque el usuario todavía debe estar pendiente del entorno.

9. Referencias

SparkFun Electronics. (s.f.). *Pull-up resistors*. SparkFun Learn. Recuperado el 28 de septiembre de 2026, de https://learn.sparkfun.com/tutorials/pull-up-resistors/all

The Engineering Projects. (2017, julio). *Introduction to L293D*. Recuperado el 28 de septiembre de 2026, de https://www.theengineeringprojects.com/2017/07/introduction-to-l293d.html